Dreamer from Pixels: Midterm Baseline and Reproduction Plan

Chevy Yang
chenyiy@andrew.cmu.edu

Zhiyuan Yang
dyang3@andrew.cmu.edu

Team 25
TA Mentor: Kangping Liu

Abstract

We study visual control with Dreamer V1 in the DeepMind Control Suite. We use Julius Frost's PyTorch implementation, fix the source commit, and test the pipeline on `cartpole_swingup` before extending to other tasks. The completed baseline uses 64 x 64 RGB pixels, action repeat 2, one training seed, and 100,000 environment steps. A ten-update gate changed the world model, actor, and critic with finite diagnostics. The final policy reached mean episodic return 181.701 on ten fixed environment seeds, compared with 70.684 for the same model before training and 55.540 for a random policy. These results are initial evidence from one bounded run. A separate long-run attempt has only partial progress evidence through 144,000 environment steps; its final checkpoint and evaluation were not recovered. We still need matched multi-seed runs and the two proposal experiments before drawing stronger conclusions.

1 What We Want to Study

Visual control requires an agent to learn both a useful representation and a policy from raw images. Dreamer addresses this problem with a learned latent dynamics model and policy improvement inside imagined trajectories. Our main goal is to reproduce and understand this Dreamer V1 pipeline from pixels. We start with `cartpole_swingup`, then plan to evaluate `walker_walk` and `ball_in_cup_catch` after the baseline is stable.

We evaluate control with episodic return and learning curves. We do not use classification accuracy because the task is continuous control. We use a single training seed for the current pilot and keep the evaluation protocol separate from the training interaction. The current report is an initial baseline report, not a claim that we have matched the paper's full experimental study.

2 Background and Related Work

PlaNet introduced recurrent latent dynamics for planning from pixels [4]. Dreamer follows the same general direction but replaces repeated online planning with value learning through latent imagination: it rolls the actor forward in the learned model and sends return gradients back through the imagined dynamics [3]. We use Dreamer V1 as the baseline and cite PlaNet to explain the predecessor relationship; we do not combine their results or implementations.

We use Julius Frost's PyTorch implementation of Dreamer V1 at commit `ccea6ae4a397a94c328891bd78e81d52dd156cb6`, with the pinned `rlpyt` submodule at `f04f23db1eb7b5915d88401fca67869968a07a37` [1]. The DeepMind Control Suite provides the simulated continuous-control task and reward contract [5]. We preserve the upstream MIT

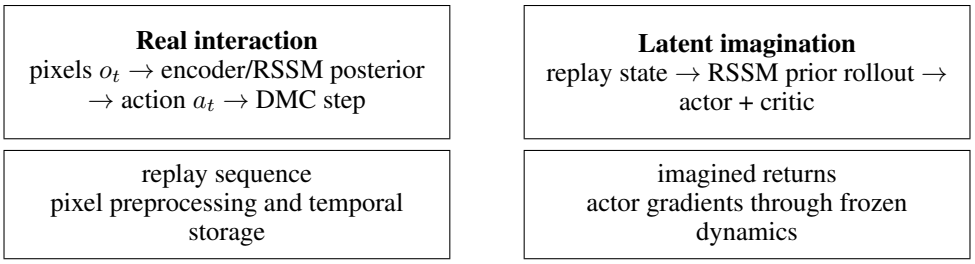


Figure 1: Our simplified Dreamer data flow. Real interaction supplies image sequences to replay. Training starts from replay states and uses the learned RSSM for latent imagination; the critic supplies return targets and actor gradients pass through the imagined dynamics.

Table 1: Effective configuration of the 100k-environment-step run.

Task / observation	cartpole_swingup, camera 0, 64 x 64 RGB
Training seed	0
Decision / environment budget	50,000 / 100,000
Action repeat / episode limit	2 / 500 decisions (1,000 env steps)
Prefill	1,000 decisions (2,000 env steps)
Training schedule	every 1,000 decisions, 100 updates
Batch size / length	16 / 50
Replay capacity	60,000 decision transitions
Precision / `use_pcont`	32-bit / False
Evaluation environment seeds	1000–1009

license and record our wrapper and measurement changes separately. The original Google Research repository is retained as a historical Dreamer V1 software reference, not as a second implementation used in this run [2].

3 Method

3.1 Dreamer path

The agent stores image sequences generated by real environment interaction. The world model encodes observations and infers a recurrent stochastic state. The actor and critic then operate on latent states imagined from replay states. Figure 1 summarizes the two paths.

The environment wrapper renders camera 0 as a 64 x 64 RGB image and stores it as a uint8 tensor in channel-first format. Before the encoder, the implementation converts pixels from $[0, 255]$ to $[-0.5, 0.5]$. One action is repeated for two underlying environment steps. Thus, 50,000 policy decisions correspond to 100,000 environment steps. Episodes contain 500 decisions, or 1,000 environment steps. The replay buffer stores temporal sequences, and the training batch has 16 sequences of length 50.

3.2 Objectives used in this run

The world model learns an observation reconstruction distribution, a reward distribution, and an RSSM prior/posterior agreement term. The implementation also supports a continuation-probability head, but our formal run uses `use_pcont=False`. Therefore, the continuation head is not constructed, its loss is not included in the world-model objective, and the imagined-return calculation uses the fixed discount factor instead. The logged `pcont_loss` field is a zero placeholder when this option is disabled; it is not evidence that a continuation loss was trained.

The actor is trained on returns imagined from posterior states, while the critic predicts those imagined returns. The main checks for the debug gate were finite losses and gradients, nonzero changes to all three parameter groups, and loading the saved model in a new process. We do not claim full optimizer/replay/environment-state continuation from the exported checkpoint.

Table 2: Two pilot configurations. Budget, replay, and initial-checkpoint handling changed together; this is not a one-factor ablation.

	6k pilot	100k run
Decision / environment steps	3,000 / 6,000	50,000 / 100,000
Replay capacity (decision transitions)	1,060	60,000
Batch size x length	16 x 50	16 x 50
Train every / updates	1,000 / 100	1,000 / 100
Initial comparison	short-run snapshot	formal pre-update snapshot
Final Dreamer mean return	115.463	181.701

Table 3: Evaluation on ten episodes for each policy, using environment seeds 1000–1009.

Policy	Mean return	Episode sample std.
Random	55.5404	42.5973
Training-initial	70.6835	3.1504
Final Dreamer	181.7011	27.5705

4 Baseline and Experimental Iteration

We first tried the historical TensorFlow route, but the available Python/Keras environment exposed old API and MuJoCo dependency failures. We then used the pinned PyTorch source in the already working Kaggle PyTorch environment. A fresh child process passed the CPU `dm-control` import/reset probe, and the GPU path passed CUDA, EGL pixel rendering, and the short Dreamer update gate. We reused the validated `dm-control -no-deps` route rather than changing the algorithm to fit a new dependency stack. These environment changes matter for reproducibility, but they are not separate scientific experiments.

Before the longer run, we corrected an intermediate configuration with `train_every=1` and `train_steps=8`; the formal run restored the upstream-style schedule of 1,000 decisions and 100 updates. We also increased replay capacity from the short pilot's 1,060 decision transitions to 60,000. The initial comparison is a snapshot of the formal training model after prefill and before its first optimizer update, not a separately initialized policy.

5 Results

The ten-update gate changed the world model, actor, and critic by maximum absolute parameter differences of 0.011696, 0.000710, and 0.000771. The formal run completed 50,000 decisions, 100,000 environment steps, and 4,900 optimizer updates. The raw checkpoint, metrics, progress file, configuration, and evaluation CSV were downloaded and checked. A later independent seed-0 long-run attempt has a locally verified intermediate progress file ending at 72,000 decisions, 144,000 environment steps, and 7,100 updates. We did not recover its final configuration, checkpoint, evaluation CSV, summary, or export archive, so we keep it as partial progress and do not append it to the 100k curve.

We evaluated three policies on the same ten environment seeds, 1000–1009. Each policy therefore has ten episodes; these are not ten training seeds. Training seed 0 is the only training seed in this report. The reported $\pm$ values are sample standard deviations across evaluation episodes. We controlled the environment seed, but we do not claim that every source of evaluation randomness was fixed.

The final mean return is higher than the training-initial snapshot and the random control in this pilot. The theoretical maximum episodic return is 1,000, so 181.7 is still a limited absolute score. Return is not success rate. The final policy also has visible episode-to-episode variation, so the mean alone is not a complete description.

Figure 2 shows two different measurements. The left panel is the training `ReturnAverage` from rlpyt: an average over up to the 100 most recently completed training episodes at each logging point. It is a windowed training diagnostic, not an evaluation learning curve. It rises from roughly the low 70s near the prefill boundary to roughly 120 near the end, with fluctuations after the early increase.

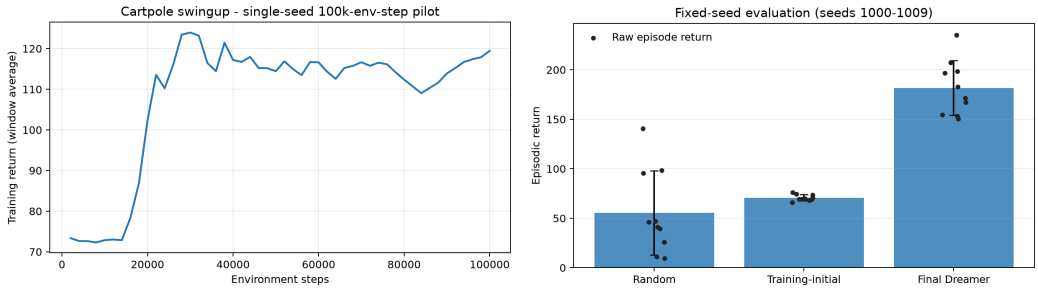


Figure 2: Left: rlpyt training-return window average versus environment steps. The x-axis is environment steps; the underlying log records decision steps and action repeat is 2. Right: fixed-seed evaluation; black points are the ten raw episode returns for each policy and error bars are the sample standard deviation across those ten episodes.

Training uses exploration noise, so this curve should not be read as the same quantity as the final evaluation table. The right panel shows all raw evaluation episode returns as points, together with the mean and the sample standard deviation.

6 Limitations and Future Work

The current evidence comes from one task, one training seed, and a bounded budget. We do not claim statistical significance, stable task mastery, or paper-level performance. The plots do not identify a specific failure behavior, and we do not infer that low returns are caused by a particular code bug or only by insufficient training. The checkpoint was loaded for evaluation, but full restoration of optimizer, replay, and environment state remains untested. The later long-run attempt is retained as an artifact-persistence failure case: its partial progress and logs are available, but no recovered final evaluation supports a performance claim.

Our next steps follow the proposal. First, we will inspect evaluation videos, the fixed-seed protocol, and the training curve to decide whether the baseline should run longer. Second, we will add training seeds 1 and 2 with the same budget and configuration. Third, we will run matched-budget sensitivity experiments for the proposal's two directions: (i) imagination horizons such as 5, 15, and 30, which are candidate settings and have not been executed; and (ii) a 4-pixel random-shift image augmentation. For the random-shift study, we will specify padding and crop behavior before implementation, keep the transform consistent within each replay sequence, and document how the encoder input and decoder reconstruction target are transformed. We will not assume that augmentation improves performance.

7 Conclusion

We now have a working Dreamer V1 training and evaluation pipeline for pixel-based `cartpole_swingup`. The formal run changed all three learned modules and produced a higher final mean return than its pre-update snapshot under the same ten evaluation environment seeds. This is useful baseline evidence, but replication still requires matched additional training seeds, protocol checks, and the two proposal experiments.

Team Contributions (Pending Confirmation)

Chevy Yang and Zhiyuan Yang both contributed to the seed-0 Dreamer V1 baseline, including experiment setup, execution, and review of the resulting evidence. The report treats this as one shared training run, not as independent training seeds. The finer division of implementation, artifact preparation, and analysis will be finalized after the team confirms the remaining details.

GitHub

Project repository: `https://github.com/DDaniel-code/dreamer-pixels`. The repository records the selected source commit, configuration differences, raw-artifact manifest, and report materials.

References

[1] Julius Frost. Dreamer pytorch. GitHub repository, n.d. URL `https://github.com/juliusfrost/dreamer-pytorch`. MIT-licensed source; fixed commit is recorded in the report text.

[2] Google Research. Dreamer: Learning behaviors by latent imagination. GitHub repository, n.d. URL `https://github.com/google-research/dreamer`. Historical Dreamer V1 reference implementation.

[3] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019. URL `https://arxiv.org/abs/1912.01603`. 2019 arXiv preprint; ICLR 2020 conference version.

[4] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pages 2555–2565. PMLR, 2019. URL `https://proceedings.mlr.press/v97/hafner19a.html`.

[5] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018. URL `https://arxiv.org/abs/1801.00690`.